

Jump Statement in C

Jump statements are used in C or C++ for the unconditional flow of control throughout the functions in a program.

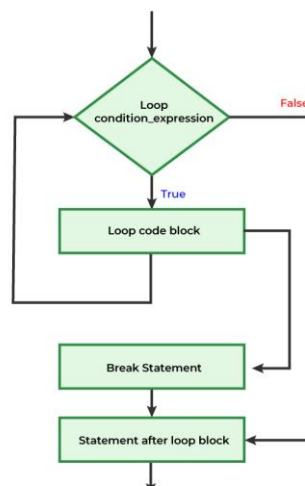
They support four types of jump statements:

- Break
- Continue
- Goto
- Return

➤ Break

- This loop control statement is used to terminate the loop. As soon as the break statement is encountered from within a loop, the loop iterations stop there, and control returns from the loop immediately to the first statement after the loop.
- Basically, break statements are used in situations when we are not sure about the actual number of iterations for the loop or we want to terminate the loop based on some condition.

Break Statement Flow Diagram



```
#include <stdio.h>
void findElement(int arr[], int size, int key)
{
    // loop to traverse array and search for key
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            printf("Element found at position: %d",
                (i + 1));
            break;
        }
    }
}

int main()
{
    int arr[] = { 1, 2, 3, 4, 5, 6 };

    // no of elements
    int n = 6;

    // key to be searched
    int key = 3;

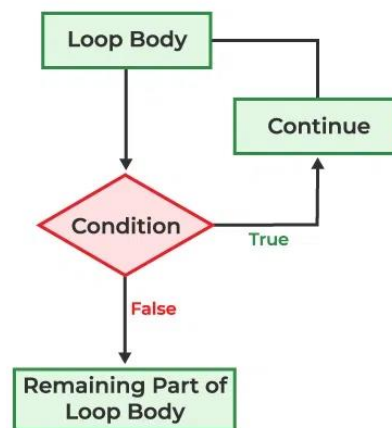
    // Calling function to find the key
    findElement(arr, n, key);

    return 0;
}
```

Output: Element found at position: 3

➤ Continue

- This loop control statement is just like the break statement.
- The continue statement is opposite to that of the break *statement*, instead of terminating the loop, it forces to execute the next iteration of the loop.
- As the name suggests the continue statement forces the loop to continue or execute the next iteration.
- When the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped and the next iteration of the loop will begin.



```
#include <stdio.h>

int main()
{
    // loop from 1 to 10
    for (int i = 1; i <= 10; i++) {

        if (i == 6)
            continue;

        else
            // otherwise print the value of i
            printf("%d ", i);
    }
}
```

```
return 0;
}
```

Output: 1 2 3 4 5 7 8 9 10

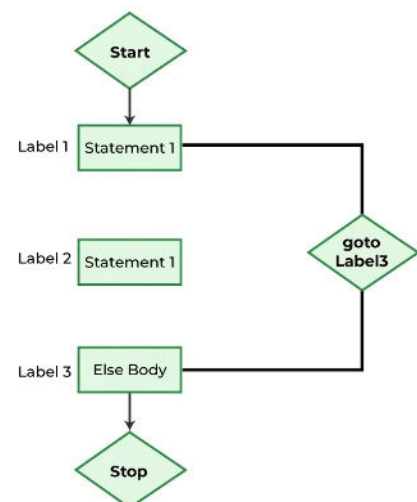
➤ Goto

- The goto statement in C/C++ also referred to as the unconditional jump statement can be used to jump from one point to another within a function.

Syntax1		Syntax2

goto label;		label:
.		.
.		.
.		.
label:		goto label;

- In the above syntax, the first line tells the compiler to go to or jump to the statement marked as a label.
- Here, a label is a user-defined identifier that indicates the target statement. The statement immediately followed after 'label:' is the destination statement.
- The 'label:' can also appear before the 'goto label;' statement in the above syntax.

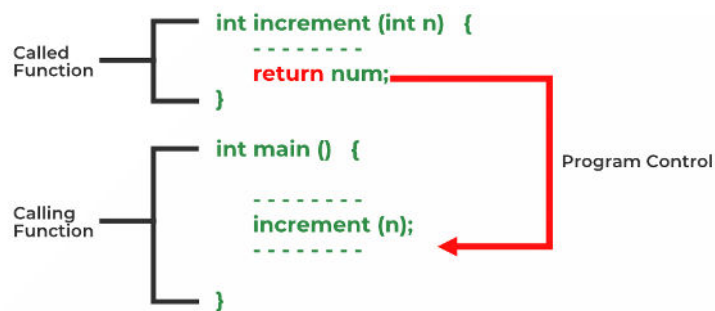


```
#include <stdio.h>
// function to print numbers from 1 to 10
void printNumbers()
{
    int n = 1;
label:
    printf("%d ", n);
    n++;
    if (n <= 10)
        goto label;
}
// Driver program to test above function
int main()
{
    printNumbers();
    return 0;
}
```

Output1 2 3 4 5 6 7 8 9 10

➤ Return

- The return in C or C++ returns the flow of the execution to the function from where it is called. This statement does not mandatorily need any conditional statements.
- As soon as the statement is executed, the flow of the program stops immediately and returns the control from where it was called.
- The return statement may or may not return anything for a void function, but for a non-void function, a return value must be returned.



```
#include <stdio.h>

int SUM(int a, int b)
{
    int s1 = a + b;
    return s1;
}

void Print(int s2)
{
    printf("The sum is %d", s2);
    return;
}

int main()
{
    int num1 = 10;
    int num2 = 10;
    int sum_of = SUM(num1, num2);
    Print(sum_of);
    return 0;
}
```

Output: The sum is 20